
G-RANS: Generalizable Residual-Aware Neural Solvers for Sparse Systems

Weixin Liao^{*1,2} Mingquan Feng^{*1,2} Zhizhou Zhang² Youjia Wu² Yifan Fu¹ Junchi Yan¹

Abstract

Neural operators have shown promise in accelerating PDE solvers, yet they remain unreliable for the sparse linear systems induced by discretization due to limited generalization across physical parameters and insufficient accuracy, and hybrid neural iterative schemes face stagnation as the residual distribution evolves over iterations. To address these limitations, we propose G-RANS (Generalizable Residual-Aware Neural Solver), a neuralized iterative paradigm that performs residual-aware subspace corrections by mapping the residual to the matrix graph, generating multi-scale correction subspaces via a residual-aware basis generator, and applying projected updates through a differentiable subspace projection. G-RANS is trained fully self-supervised via progressive bootstrap with multi-stage residual distributions. On sparse FEM systems from a representative suite of second-order elliptic PDEs (Poisson, advection–diffusion, reaction–diffusion, and Helmholtz), G-RANS is robust to severe coefficient shifts (up to $\pm 70\%$) and shows strong cross-equation generalization. G-RANS reaches relative residuals on the order of 10^{-5} , substantially outperforming end-to-end neural operators, and attains the same target residual with 2–4 $\times$ less wall-clock time than classical Krylov solvers.

1. Introduction

Neural operators have emerged as a powerful method for accelerating scientific computing by learning end-to-end mappings between function spaces (Kovachki et al., 2023). They achieve impressive performance on surrogate pre-

diction for PDE solutions and have been widely adopted in simulation pipelines. However, in many high-impact applications—including high-resolution fluid dynamics and electromagnetic wave propagation—the dominant cost of a high-fidelity solve is not the PDE evaluation itself but the large, sparse linear system $\mathbf{Ax} = \mathbf{b}$ induced by discretization. As resolution increases, these systems quickly become the computational bottleneck, making scalable and reliable sparse-system solvers an obstacle to practical simulation.

In this setting, purely end-to-end neural prediction is often unreliable: it may generalize poorly under parameter shifts, and its single-pass accuracy can be insufficient for stringent accuracy requirements. A complementary line of work, therefore, integrates learning modules into classical iterative solver structures. For sparse systems, state-of-the-art high-accuracy methods are dominated by subspace correction schemes, most notably Krylov solvers such as GMRES (Watkins, 2007; Saad, 2003), which motivates neural solvers that learn to construct effective correction subspaces. A central obstacle, however, is that residuals encountered during solving undergo substantial distribution shift across iterations, leading to what we term the **stagnation problem**.

For example, a weakened variant of our method without residual-aware design (sub-NSS) exhibits the typical stagnation behavior: after roughly 7–8 correction steps, its per-iteration residual reduction drops below 5% and progress largely plateaus (Fig. 1-Left; see Sec. 4). Consequently, many recent hybrid approaches focus on neural preconditioning while still relying on classical Krylov iterations to reach high accuracy (Luo et al., 2024; Chen, 2025).

We propose **G-RANS** (Generalizable Residual-Aware Neural Solver), a neuralized subspace-solver for high-fidelity sparse systems via iterative residual-aware corrections, mitigating the stagnation that arises as residual distributions shift over iterations. To exploit residual information on sparse systems, G-RANS represents the current residual on a matrix graph as a residual-aware graph representation. At each iteration, a *Residual-Aware Basis Generator* (RBG) produces a block of multi-scale correction directions, instantiated by a multi-scale GATv2 (Brody et al., 2022) backbone for efficient representation, and a differentiable numerical subspace projection (QR + least squares) then computes the residual-minimizing update within the predicted correction

^{*}Equal contribution ¹Sch. of Artificial Intelligence & Sch. of Computer Science, Shanghai Jiao Tong University, China ²Bosch (China) Invest Ltd., Shanghai, China. Correspondence to: Junchi Yan <yanjunchi@sjtu.edu.cn>. This work was partly supported by Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China, JYB2025XDXM411.

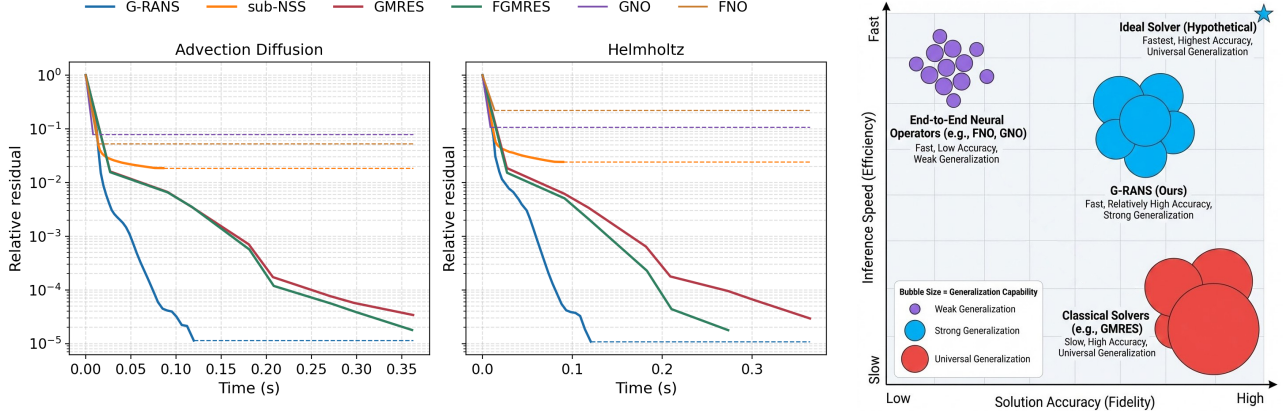


Figure 1. Left: time-residual trade-off on representative sparse FEM systems ($n \approx 1000$), including a nonsymmetric advection–diffusion system and an indefinite Helmholtz system in our experiments, showing that G-RANS sustains iterative residual reduction and reaches the 10^{-5} relative-residual regime faster than Krylov baselines. In contrast, end-to-end neural operators plateau at much higher residuals, and a neural subspace solver without residual-aware design (sub-NSS, see Sec. 4) exhibits the typical stagnation behavior of neural iterative schemes. **Right:** a *conceptual* diagram situating G-RANS among solver paradigms. Axes reflect typical speed–accuracy trade-offs, and bubble size qualitatively represents generalization capability; the plot is only an illustrative schematic.

Table 1. Recent neural work on sparse linear systems $\mathbf{Ax} = \mathbf{b}$ primarily learns *initializations/accelerators* for classical iterations or learns *preconditioners*, still relying on numerical solvers as the accuracy backbone. In contrast, G-RANS is a standalone neural solver that performs iterative, residual-aware subspace corrections. CG-based variants(e.g. (Kaneda et al., 2023)) typically require $\mathbf{A}$ to be symmetric positive definite (SPD), whereas Krylov/subspace-based approaches apply more broadly; G-RANS does not assume specific matrix properties and supports nonsymmetric and indefinite systems. Moreover, many learning-based accelerations require extra supervision (e.g., eigenvectors(Luo et al., 2024) or solution pairs(Chen, 2025)), while G-RANS is trained fully self-supervised. See Appendix A for details.

Method	Position	Assumption on $\mathbf{A}$	Learned functionality	Supervision	Extra labels
G-RANS (ours)	Neural solver	No SPD assumption	Residual-adaptive basis	Self-supervised	None
NeurKitt (Luo et al., 2024)	Krylov acceleration	No SPD assumption	Invariant subspace	Supervised	Eigenvectors
DCDM (Kaneda et al., 2023)	CG direction predictor	Requires SPD	Approximate inverse	Supervised	Rayleigh–Ritz vectors
GNP (Chen, 2025)	Neural preconditioner	No SPD assumption	Preconditioner	Supervised	Solution pairs $(\mathbf{b}, \mathbf{x})$
NP-KSG (Dimola & Coclite, 2025)	Neural preconditioner	Assumed on SPD	Preconditioner	Self-supervised	None

subspace. To match this residual-aware iterative paradigm, we train G-RANS fully self-supervised via a multi-stage progressive bootstrap curriculum, where residuals generated along the solver trajectory are recycled as training inputs to induce a native multi-stage residual distribution.

By theoretical analysis, we relate G-RANS-style subspace correction to the residual-reduction mechanism of Krylov solvers, showing that the induced subspace iteration can achieve GMRES-comparable residual decrease in the appropriate regime (see Sec. 3).

We evaluate on sparse FEM systems from four representative families of 2nd-order elliptic PDEs (Poisson, advection–diffusion, reaction–diffusion, and Helmholtz) at problem sizes $n \approx 1K$ and $n \approx 2K$, where G-RANS can reduce the relative residual to the 10^{-5} level, remains robust under severe coefficient shifts (up to $\pm 70\%$), and exhibits strong cross-equation generalization within this class, notably outperforming end-to-end neural operators (GNO (Li et al., 2020) and FNO (Li et al., 2021)) in accuracy.

Within the accuracy regime reached by G-RANS, it consistently solves faster than Krylov baselines (GMRES, FGMRES, and MINRES (Saad, 2003)), yielding $2\text{--}4\times$ wall-clock speedups across experiments (Fig. 1-Left). Moreover, G-RANS is compatible with standard preconditioning. In experiments with Jacobi preconditioning, it remains faster than the Krylov solvers. Specifically, we categorize the related work in three groups with a key summary in Table 1: i) Classical Iterative Solvers and Preconditioning; ii) Fixed Subspace Initialization and Neural Warm Starts; iii) Learned Preconditioning for Krylov Methods. See the detailed discussion in Sec. A in the Appendix. **The contributions are:**

i) To our best knowledge, we present the first neural iterative subspace-solver framework that solves a broad class of sparse linear systems arising from PDE discretization, covering both nonsymmetric and indefinite regimes, and achieves high-fidelity accuracy as a neural solver.

ii) G-RANS exhibits strong generalization across substantial coefficient variations and across equation types within a representative family of PDEs.

iii) For high-throughput sparse solving tasks with moderate accuracy requirements ($\approx 10^{-5}$ relative residual), G-RANS can act as an alternative to classical iterative solvers to improve wall-clock efficiency.

2. Methodology

As summarized in Fig. 2, our method is built around three components: (i) a residual-base graph representation that naturally aligns with sparse matrices while explicitly modeling the current residual together with geometric positional encoding, (ii) multi-scale subspace prediction that produces a block of correction directions from the current residual, and (iii) a differentiable projection step that turns the predicted directions into a residual-minimizing update, enabling end-to-end training through solver iterations. In addition, we adopt a multi-stage progressive bootstrap curriculum so that residuals generated at successive correction stages form the training distribution, mitigating stagnation under residual shift.

2.1. Residual-Base Graph Formulation

Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be sparse and $\mathbf{b} \in \mathbb{R}^n$. We maintain an iterate x_t and residual

$$\mathbf{r}_t = \mathbf{b} - \mathbf{A}\mathbf{x}_t. \quad (1)$$

Denote $\mathbf{A}$ as a directed graph $G = (\mathcal{V}, \mathcal{E})$ with $|\mathcal{V}| = n$ and

$$(i, j) \in \mathcal{E} \iff A_{ij} \neq 0, \quad (2)$$

so each nonzero entry corresponds to a directed edge. Diagonal nonzeros A_{ii} appear as self-loop edges. Each edge carries a scalar attribute $e_{ij} = A_{ij}$.

At iteration t , node i receives a residual-aware input coupled with geometry. Let coords_i denote the physical coordinate. We encode coordinates using a learnable positional encoder ψ_{pos} (MLP) and concatenate it with the residual component:

$$\mathbf{p}_i = \psi_{\text{pos}}(\text{coords}_i), \quad \mathbf{z}_i^{(t)} = [\tilde{\mathbf{r}}_{t,i} \parallel \mathbf{p}_i]. \quad (3)$$

To stabilize scale across instances and across iterations, we normalize the residual per graph before message passing:

$$\tilde{\mathbf{r}}_t = \frac{\mathbf{r}_t}{\|\mathbf{r}_t\|_2 + \varepsilon}, \quad (4)$$

applied only to the residual channel.

2.2. Residual-Aware Basis Generator (RBG)

Given the residual-aware matrix graph representation $(G, \{z_i^{(t)}\}, \{e_{ij}\})$, the Residual-Aware Basis Generator (RBG) outputs a block of candidate basis vectors

$$\mathbf{V}_t = \text{RBG}_\theta(\mathbf{A}, \mathbf{r}_t, \text{coords}) \in \mathbb{R}^{n \times k}, \quad (5)$$

which will be converted into a correction subspace for differentiable projection in Sec. 2.3. RBG is designed with two requirements: *residual-awareness* (the basis should adapt to the current residual state on the operator graph) and *multi-scale propagation* (the basis should capture correction directions induced by different propagation radii).

Design choice. We instantiate RBG with an improved GAT network architecture (We refer to this network as Poly-GAT), which is a concrete polynomial-style instantiation built upon a multi-scale Graph Attention Network v2 (GATv2) backbone (Brody et al., 2022; Veličković et al., 2018). Specifically, Poly-GAT leverages layered graph neural network embeddings to emit distinct blocks of candidate basis vectors across different propagation scales, capturing multi-hop polynomial features of the sparse matrix operator directly conditioned on the current residual state. This choice is motivated by (i) its ability to perform efficient multi-hop propagation on sparse matrix graphs using the edge attribute $e_{ij} = A_{ij}$, and (ii) its compatibility with our theoretical results, which show that Poly-GAT combined with subspace projection can recover GMRES behavior under a specific configuration (Sec. 3).

Propagation on matrix graph. Denote $h_i^{(\ell)}$ the feature of node i at layer ℓ and $\mathcal{N}_i$ its neighbors in matrix graph. Each layer performs coefficient-aware neighborhood aggregation,

$$h_i^{(\ell+1)} = \sum_{j \in \mathcal{N}_i} \gamma_{ij}^{(\ell)} \mathbf{W}^{(\ell)} h_j^{(\ell)}, \quad (6)$$

where the mixing weights $\gamma_{ij}^{(\ell)}$ are produced by a learnable function of local node states and the edge attribute $e_{ij} = A_{ij}$. The initial feature $h^{(0)}$ is constructed from the residual-aware node input $z_i^{(t)}$ (Sec. 2.1), so the propagation is explicitly conditioned on the current residual.

Multi-scale basis emission. To extract correction directions at multiple propagation scales, Poly-GAT emits basis blocks from multiple sequential layers. Let $\mathbf{h}^{(\ell)} \in \mathbb{R}^{n \times d}$ denote the node embeddings at depth ℓ , where d is the hidden dimension. A lightweight linear head attached to each layer $\ell \in \{1, \dots, L\}$ projects these embeddings to emit b candidate directions:

$$\mathbf{B}_t^{(\ell)} = \text{Head}^{(\ell)}(\mathbf{h}^{(\ell)}) \in \mathbb{R}^{n \times b}, \quad (7)$$

where L represents the total number of layers, and b is the number of basis vectors generated per layer. We then form the complete candidate basis matrix $\mathbf{V}_t \in \mathbb{R}^{n \times k}$ by concatenating the emissions across all depths:

$$\mathbf{V}_t = [\mathbf{B}_t^{(1)} \parallel \mathbf{B}_t^{(2)} \parallel \dots \parallel \mathbf{B}_t^{(L)}], \quad k = L \cdot b. \quad (8)$$

where k denotes the total dimension of the newly generated subspace block at iteration t .

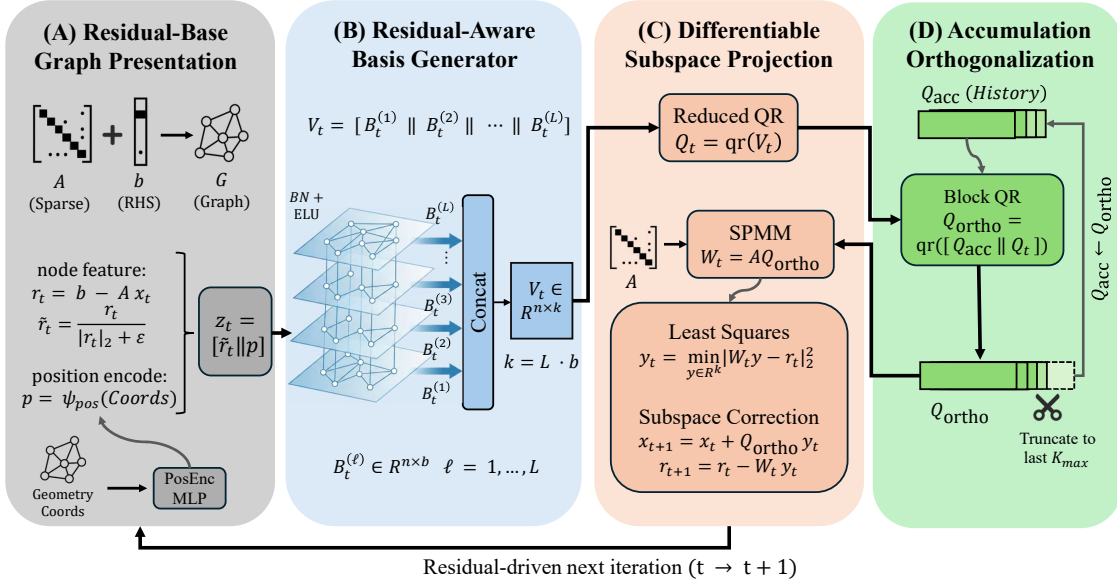


Figure 2. **G-RANS**. Each iteration builds a residual-adaptive correction subspace via Residual-Aware Basis Generator (RBG; instantiated with a multi-scale GATv2 backbone), then applies a differentiable numerical subspace projection to update solution & residual.

2.3. Differentiable Numerical Subspace Projection

Given the candidate basis matrix $V_t \in \mathbb{R}^{n \times k}$, G-RANS computes an orthogonalized subspace block via a reduced QR factorization to stabilize the numerical projection:

$$Q_t = \text{qr}(V_t) \in \mathbb{R}^{n \times k}, \quad (9)$$

where Q_t comprises k orthonormal columns spanning the identical subspace as $\text{span}(V_t)$.

We then compute a residual-minimizing update within the subspace by solving a least-squares problem:

$$W_t = A Q_t, \quad (10)$$

$$y_t = \arg \min_{y \in \mathbb{R}^k} \|W_t y - r_t\|_2. \quad (11)$$

The solution and residual are updated as

$$x_{t+1} = x_t + Q_t y_t, \quad r_{t+1} = r_t - W_t y_t. \quad (12)$$

The projection is numerical (QR + least squares) and differentiable (Golub & Van Loan, 2013), allowing gradients to backpropagate from the residual objective to θ through the solver loop.

2.4. Subspace Accumulation via Block Re-orthogonalization

Repeated residual-aware corrections require the active search space to remain informative while preventing the catastrophic loss of orthogonality across iterative steps. Instead of projecting exclusively onto the newly generated block $\text{span}(Q_t)$, we maintain a historical orthonormal basis

matrix $Q_{\text{acc}} \in \mathbb{R}^{n \times k_{\text{acc}}}$ that stores the directional history from previous steps. We then form a comprehensive, joint projection basis Q_{ortho} via block re-orthogonalization:

$$Q_{\text{ortho}} = \text{qr}([Q_{\text{acc}} \parallel Q_t]). \quad (13)$$

To strictly bound the memory footprint and the computational complexity of the subsequent dense linear algebra, we implement a sliding window governed by a hyperparameter K_{max} . If the number of columns in Q_{ortho} exceeds K_{max} , we truncate Q_{ortho} to retain only the most recent K_{max} columns.

$$Q_{\text{ortho}} \leftarrow \begin{cases} Q_{\text{ortho}}, & \text{if } \text{cols}(Q_{\text{ortho}}) \leq K_{\text{max}}, \\ Q_{\text{ortho}}[:, -K_{\text{max}}:], & \text{otherwise.} \end{cases} \quad (14)$$

The projection in Sec. 2.3 is then performed with $\text{span}(Q_{\text{ortho}})$. After computing the solution, the historical memory is carried forward by updating $Q_{\text{acc}} \leftarrow Q_{\text{ortho}}$ for the next iteration.

$$Q_{\text{acc}} \leftarrow Q_{\text{ortho}}. \quad (15)$$

Operationally, block re-orthogonalization amortizes orthogonalization over basis blocks, enabling substantially larger active subspaces than per-vector updates.

2.5. Multi-stage Progressive Bootstrap Training

A key challenge in neural iterative solvers is residual distribution shift: residuals encountered after one or more corrections differ from the initial residuals, which can lead to stagnation if training only covers early-stage inputs.

Algorithm 1 G-RANS inference

```

1: Input: Sparse matrix  $\mathbf{A}$  (directed edges with attributes  $A_{ij}$ ), RHS  $\mathbf{b}$ , coordinates  $\{\text{coords}_i\}$ , maximum iterations  $T_{\max}$ , window size  $K_{\max}$ ;
2: Output: Solution  $\mathbf{x}$ .
3:  $\mathbf{x} \leftarrow \mathbf{0}$ ,  $\mathbf{r} \leftarrow \mathbf{b}$ ,  $\mathbf{Q}_{\text{acc}} \leftarrow []$ .
4: for  $t = 1$  to  $T_{\max}$  do
5:   Normalize residual  $\tilde{\mathbf{r}} \leftarrow \mathbf{r}/(\|\mathbf{r}\|_2 + \varepsilon)$  and form node inputs  $\mathbf{z}_i = [\tilde{\mathbf{r}}_i]_{\psi_{\text{pos}}(\text{coords}_i)}$ .
6:    $\mathbf{V} \leftarrow \text{RBG}_{\theta}(\mathbf{A}, \mathbf{r}, \text{coords})$ 
7:    $\mathbf{Q}_t \leftarrow \text{qr}(\mathbf{V})$  (intra-block QR)
8:    $\mathbf{Q}_{\text{ortho}} \leftarrow \text{qr}([\mathbf{Q}_{\text{acc}} \parallel \mathbf{Q}_t])$  (projection basis)
9:    $\mathbf{Q}_{\text{ortho}} \leftarrow \text{TruncateToLast}(\mathbf{Q}_{\text{ortho}}, K_{\max})$  (sliding window)
10:   $\mathbf{W} \leftarrow \mathbf{A}\mathbf{Q}_{\text{ortho}}$  (sparse–dense multiply)
11:   $\mathbf{y} \leftarrow \arg \min_{\mathbf{y}} \|\mathbf{W}\mathbf{y} - \mathbf{r}\|_2$  (least squares)
12:   $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{Q}_{\text{ortho}}\mathbf{y}$ ,  $\mathbf{r} \leftarrow \mathbf{r} - \mathbf{W}\mathbf{y}$  (projected update)
13:   $\mathbf{Q}_{\text{acc}} \leftarrow \mathbf{Q}_{\text{ortho}}$  (carry history)
14:  if converged or stagnated then
15:    break
16:  end if
17: end for
18: return  $\mathbf{x}$ 
    
```

We use a progressive bootstrap curriculum that trains G-RANS on a multi-stage residual distribution generated by its own solver trajectory. Training is self-supervised: given $(\mathbf{A}, \mathbf{b}, \text{coords})$, the loss minimizes residual norms after projection.

Let $\mathbf{x}_0 = \mathbf{0}$ and $\mathbf{r}_0 = \mathbf{b}$. At each step, the model produces V_{t+1} from $\mathbf{r}_t$, forms the projection basis $\mathbf{Q}_{\text{ortho}}^{(t+1)}$, solves least squares, and updates $(\mathbf{x}_{t+1}, \mathbf{r}_{t+1})$.

Stage I trains a single correction:

$$\mathcal{L}_1 = \|\mathbf{r}_0 - \mathbf{A}\mathbf{Q}_{\text{ortho}}^{(1)}\mathbf{y}_1\|_2^2. \quad (16)$$

Stage N bootstraps multi-call behavior by unrolling N steps on the same $(\mathbf{A}, \mathbf{b}, \text{coords})$ and summing per-step residual objectives, where $\mathbf{y}_{t+1} = \arg \min_{\mathbf{y}} \|\mathbf{A}\mathbf{Q}_{\text{ortho}}^{(t+1)}\mathbf{y} - \mathbf{r}_t\|_2$.

$$\mathcal{L}_N = \sum_{t=0}^{N-1} \|\mathbf{r}_t - \mathbf{A}\mathbf{Q}_{\text{ortho}}^{(t+1)}\mathbf{y}_{t+1}\|_2^2, \quad (17)$$

To stabilize optimization across stages, we treat the intermediate residuals produced by earlier corrections as fixed bootstrap inputs, so gradients do not backpropagate through their generation, while keeping the projection step differentiable to preserve a direct learning signal for basis generation.

Also, we train each stage for a sufficient number of epochs before advancing, ensuring that the bootstrapped residuals

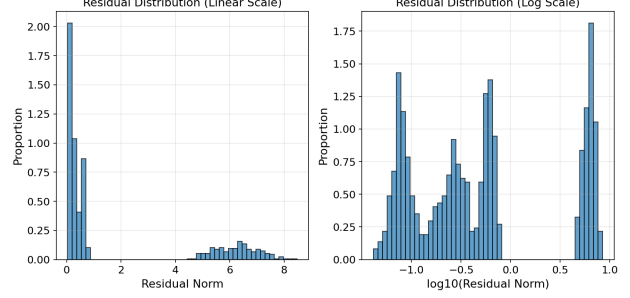


Figure 3. Relative Residual distribution of training data induced by the three-stage progressive bootstrap curriculum. Left: histogram in linear scale. Right: histogram in log scale, revealing three well-separated modes corresponding to residual regimes produced at successive correction stages.

are representative and correspond to progressively tighter correction targets. Balancing training cost and effectiveness, we adopt a three-stage curriculum in all experiments.

Fig. 3 visualizes the resulting Relative Residual distribution collected from the three training stages: the log-scale histogram exhibits three well-separated modes, indicating that each stage is trained to produce a distinct residual regime rather than collapsing to a single scale. This separation supports the intended role of progressive bootstrap in exposing the solver to residual distributions encountered at different stages of iterative refinement.

2.6. Inference and Computational Cost

Algorithm 1 summarizes G-RANS at test time. We initialize $\mathbf{x}_0 = \mathbf{0}$ and $\mathbf{r}_0 = \mathbf{b}$, and stop once the target relative residual is reached or when residual reduction per iteration falls below a prescribed threshold.

Let $\text{nnz}(\mathbf{A})$ be the number of nonzeros and let K be the active projection dimension bounded by $K_{\max}$. Each iteration is dominated by sparse–dense multiplication $\mathbf{A}\mathbf{Q}_{\text{ortho}}$ with cost $\mathcal{O}(\text{nnz}(\mathbf{A})K)$ and by the QR/least-squares operations whose leading terms scale as $\mathcal{O}(nK^2 + K^3)$ (Golub & Van Loan, 2013). The RBG forward pass adds a message-passing cost that scales with graph sparsity, number of layers, and embedding width.

3. Theoretical Analysis

While G-RANS employs non-linear neural components to learn residual-aware corrections, it is structurally designed to generalize classical Krylov subspace methods. In this section, we provide a theoretical guarantee that our Poly-GAT architecture, combined with the differentiable subspace projection, explicitly subsumes the GMRES algorithm. Specifically, we show that there exists a specific parameter initialization (the Linear Identity Configuration) under which

G-RANS structurally recovers the standard Krylov subspace. While this establishes the expressive capacity of our architecture to represent GMRES-like updates, we note that the highly non-linear training dynamics are not mathematically guaranteed to converge to this exact state.

3.1. G-RANS as a Generalization of GMRES

The GMRES minimizes the residual norm over the Krylov subspace $\mathcal{K}_m(\mathbf{A}, \mathbf{r}_0) = \text{span}\{\mathbf{r}_0, \mathbf{A}\mathbf{r}_0, \dots, \mathbf{A}^{m-1}\mathbf{r}_0\}$ at step m . The details of the GMRES algorithm are introduced in Appendix B. We demonstrate that the RBG (Poly-GAT) can construct this exact subspace.

Definition 3.1 (Linear Identity Configuration). Define the **Linear Identity Configuration (LIC)** of the RBG network:

1. **Linearity:** All non-linear activation functions (e.g., LeakyReLU) are replaced by identity maps.
2. **Edge-Coupling:** The GATv2 attention softmax is disabled such that the aggregation operation performs a matrix-vector multiplication. Specifically, for the edge-conditioned attention weights α_{ij} and projection matrix $\mathbf{W}$, we set weights such that $\alpha_{ij}\mathbf{W} = A_{ij}\mathbf{I}$. This is feasible because the edge attributes $e_{ij} = A_{ij}$ are explicit inputs to the attention network.
3. **Feature Initialization:** The node input feature is set to the raw residual $\mathbf{z}_i^{(t)} = \mathbf{r}_{t,i}$ (ignore positional encoding).

Theorem 3.2 (Recovery of Krylov Subspace). *Under the LIC, a Poly-GAT network with L layers generates a candidate basis matrix $\mathbf{V}_t$ whose column space spans the Krylov subspace of order $L + 1$:*

$$\text{span}(\mathbf{V}_t) = \mathcal{K}_{L+1}(\mathbf{A}, \mathbf{r}_t). \quad (18)$$

Consequently, the G-RANS update step is mathematically equivalent to one cycle of GMRES($L + 1$).

Proof. Let $\mathbf{h}^{(0)} = \mathbf{r}_t$. Under the LIC assumptions, the message passing update at layer ℓ simplifies to the sparse matrix-vector product:

$$h_i^{(\ell+1)} = \sum_{j \in \mathcal{N}_i} A_{ij} h_j^{(\ell)} \implies \mathbf{h}^{(\ell+1)} = \mathbf{A}\mathbf{h}^{(\ell)}. \quad (19)$$

By induction, the node features at layer ℓ are $\mathbf{h}^{(\ell)} = \mathbf{A}^\ell \mathbf{r}_t$. The Poly-GAT mechanism (Eq. 8) explicitly concatenates outputs from all layers $\ell = 0 \dots L$. Thus, the generated basis block is:

$$\mathbf{V}_t = [\mathbf{r}_t \parallel \mathbf{A}\mathbf{r}_t \parallel \dots \parallel \mathbf{A}^L \mathbf{r}_t]. \quad (20)$$

This matrix forms the basis for Krylov subspace $\mathcal{K}_{L+1}(\mathbf{A}, \mathbf{r}_t)$. Following Eqs. 9–12, G-RANS orthonormalizes $\mathbf{V}_t$ to obtain $\mathbf{Q}_t$ and computes $\mathbf{y}_t = \arg \min_{\mathbf{y}} \|\mathbf{A}\mathbf{Q}_t \mathbf{y} - \mathbf{r}_t\|_2$. This optimization problem is the exact variational definition of the GMRES approximation over the subspace spanned by the columns of $\mathbf{V}_t$. $\square$

3.2. Convergence Guarantee

Because the architecture of G-RANS can subsume GMRES as a special case under the LIC, its theoretical approximation capacity is lower-bounded by that of GMRES. Although empirical training trajectories discover different, non-linear parameterizations, evaluating the LIC provides a theoretical baseline for the network’s representational power. We can therefore extend classical convergence analyses to this specific configuration.

Corollary 3.3 (Convergence on Diagonalizable Matrices).

Let $\mathbf{A}$ be a diagonalizable matrix $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^{-1}$. Assume that the spectrum of $\mathbf{A}$ is contained in an ellipse $E(c, d, a)$ in the complex plane with center c , focal distance d , and major semi-axis a , which excludes the origin. Under the LIC, the residual norm achieved at the m -th step (where $m = L + 1$) satisfies:

$$\|\mathbf{r}_m\|_2 \leq \kappa_2(\mathbf{X}) \frac{C_m(a/d)}{|C_m(c/d)|} \|\mathbf{r}_0\|_2, \quad (21)$$

where C_m denotes the Chebyshev polynomial of the first kind of degree m , and $\kappa_2(\mathbf{X})$ is the condition number of $\mathbf{X}$. Furthermore, this bound can be approximated explicitly as:

$$\frac{\|\mathbf{r}_m\|_2}{\|\mathbf{r}_0\|_2} \lesssim \kappa_2(\mathbf{X}) \left(\frac{a + \sqrt{a^2 - d^2}}{c + \sqrt{c^2 - d^2}} \right)^m. \quad (22)$$

Remark 3.4. This result, adapted from Corollary 6.33 of Saad (2003), establishes that the G-RANS architecture possesses the structural capacity to guarantee convergence (provided the network parameters align with the LIC and the spectrum of $\mathbf{A}$ is clustered away from the origin).

We emphasize that this highlights the theoretical expressive power of the model, rather than claiming the empirical training dynamics will strictly converge to the LIC. In practice, the learned non-linear parameters allow G-RANS to construct generalized *super-Krylov* subspaces that adapt to the operator spectrum more efficiently than classical polynomial bases, which helps explain the accelerated convergence observed in Sec. 4.

4. Experiments

We evaluate G-RANS on sparse linear systems arising from 2D FEM discretizations of four representative families of second-order elliptic PDEs: Poisson, advection–diffusion (Adv.–Diff.), reaction–diffusion (React.–Diff.), and Helmholtz. Our experiments focus on:

Accuracy: the relative residuals achieved by G-RANS across diverse sparse systems.

Generalization: robustness to large parameter shifts and generalization across PDE types;

Table 2. G-RANS results on $n \approx 1000$ sparse FEM systems under parameter variations. We report neural iterations (Iters), relative residual $\|r_T\|_2/\|r_0\|_2$ (RelRes, with $r_0 = b$), absolute final residual norm $\|r_T\|_2$, per-instance wall-clock time (total t_{tot} and model-only t_{model}), and speedup of classical solvers measured in the accuracy reached by G-RANS (baseline time divided by G-RANS total time, ‘Spd’ denotes Speed up rate). MINRES is only applicable to symmetric systems and thus is not reported for non-symmetric cases.

PDE	Var.	Iters	RelRes	$\ r_T\ _2$	t_{tot}	t_{model}	Spd _{gmres}	Spd _{fgmres}	Spd _{minres}
Poisson	$\pm 10\%$	23.41	2.80e-05	1.7585e-04	91.30	17.20	3.03×	2.75×	2.59×
	$\pm 30\%$	23.45	3.20e-05	2.1695e-04	93.00	17.30	3.13×	2.67×	2.66×
	$\pm 70\%$	23.53	4.50e-05	2.5321e-04	92.90	17.70	3.53×	2.62×	2.83×
Adv.-Diff.	$\pm 10\%$	24.89	3.30e-05	2.0719e-04	98.90	18.50	3.02×	2.79×	—
	$\pm 30\%$	24.97	3.40e-05	2.1004e-04	100.20	19.20	3.10×	2.74×	—
	$\pm 70\%$	24.33	8.60e-05	4.4947e-04	101.60	20.00	3.60×	2.50×	—
Helmholtz	$\pm 10\%$	24.02	4.80e-05	3.8505e-04	95.70	17.80	3.21×	2.21×	—
	$\pm 30\%$	24.20	4.00e-05	3.0825e-04	98.20	17.90	3.34×	2.24×	—
	$\pm 70\%$	24.20	5.80e-05	4.4565e-04	94.90	17.40	3.55×	2.25×	—
React.-Diff.	$\pm 10\%$	23.91	2.40e-05	1.9308e-04	96.90	17.80	2.98×	2.21×	3.00×
	$\pm 30\%$	23.97	2.30e-05	2.0271e-04	94.30	17.80	3.16×	2.27×	3.13×
	$\pm 70\%$	23.83	4.30e-05	3.6854e-04	93.90	17.70	3.41×	2.21×	3.45×

Table 3. G-RANS and end-to-end neural operator baselines on $n \approx 1000$ sparse FEM systems under $\pm 10\%$ parameter variation. We additionally report the relative ℓ_2 error against a reference solution obtained by numerical solving.

PDE	Method	RelRes	$\ r\ _2$	Mean Rel ℓ_2
Poisson	GNO	8.38e-2	5.2205e-1	5.3399e-2
	FNO	7.40e-2	4.6213e-1	5.6569e-2
	G-RANS	3.20e-5	2.0755e-4	7.2600e-4
Adv.-Diff.	GNO	8.05e-2	5.0130e-1	4.5766e-2
	FNO	5.39e-2	3.3794e-1	4.3531e-2
	G-RANS	3.20e-5	1.9952e-4	9.2000e-4
React.-Diff.	GNO	1.15e-1	9.1742e-1	5.7519e-2
	FNO	2.274e-1	1.8093e+0	7.8602e-2
	G-RANS	2.50e-5	2.0890e-4	6.3000e-4
Helmholtz	GNO	1.096e-1	9.0369e-1	5.9972e-2
	FNO	2.257e-1	1.8555e+0	7.5403e-2
	G-RANS	4.00e-5	3.2071e-4	9.6800e-4

Efficiency: wall-clock efficiency over classical Krylov solvers in the accuracy regime reached by G-RANS;

Compatibility: integration with standard preconditioners, analogous to classical solver pipelines.

4.1. Problem Suite and Data Generation

We generate sparse systems $\mathbf{Ax} = \mathbf{b}$ from FEM on triangular meshes, where n denotes the number of unknowns after discretization. All PDE families share the same meshing pipeline, and we evaluate two problem sizes with target $n \in \{1000, 2000\}$ (with minor variation around the target due to mesh generation). For each instance, PDE coefficients are randomized with multiplicative perturbations under a two-region inclusion geometry, with mild randomiza-

Table 4. Cross-equation generalization of G-RANS on $n \approx 1000$ systems under $\pm 10\%$ parameter variation. Rows indicate the training PDE and columns indicate the test PDE. Values are the final relative residual $\|r_T\|_2/\|r_0\|_2$.

Train \ Test	Poisson	Adv.-Diff.	Helmholtz	React.-Diff.
Poisson	2.8×10^{-5}	3.4×10^{-5}	8.8×10^{-5}	5.2×10^{-5}
Adv.-Diff.	3.4×10^{-5}	3.3×10^{-5}	9.3×10^{-5}	5.4×10^{-5}
Helmholtz	3.0×10^{-5}	5.3×10^{-5}	4.8×10^{-5}	2.2×10^{-5}
React.-Diff.	2.1×10^{-5}	2.7×10^{-5}	3.6×10^{-5}	2.4×10^{-5}

tion of the inclusion location and radius. Unless stated otherwise, models are trained on a $\pm 10\%$ parameter-variation dataset and tested under different variations to assess parameter generalization. (See Appendix C for details) As the experimental setting, we use 1000 training instances, and report results averaged over 64 test instances.

End-to-end neural operators. We include representative end-to-end neural operators (GNO and FNO) trained on the same data distribution as G-RANS. For a fair comparison, we adopt standard hyperparameter settings for each operator family and do not tune the models to artificially match or compress parameter counts. In the $n \approx 1000$ setting, G-RANS uses 56,260 trainable parameters, whereas GNO and FNO use 483,937 and 601,825 parameters, respectively, highlighting that G-RANS attains its solver performance with substantially fewer parameters.

Classical Krylov solvers. Standard Krylov methods (GMRES, FGMRES, and MINRES when applicable) serve as classical baselines; all are implemented and benchmarked on the GPU for consistent timing. For GMRES/FGMRES, the restart parameter is selected based on our validation suite. To compare runtime in the accuracy regime reached

Table 5. Jacobi-preconditioned (left-preconditioned) experiments on the two empirically more challenging PDE families in our suite (reaction–diffusion and Helmholtz), evaluated under $\pm 10\%$ parameter variation at $n \approx 1000$ and $n \approx 2000$.

PDE	n	RelRes	$\ r_T\ _2$	t_{tot}	t_{model}	GMRES (ms)	Spd _{gmres}	FGMRES (ms)	Spd _{fgmres}
Helmholtz	1000	8.00e−05	1.29e−04	86.00	13.60	192.70	2.24×	193.50	2.25×
	2000	1.54e−03	2.62e−03	93.40	18.70	188.00	2.01×	188.70	2.02×
React.–Diff.	1000	4.30e−05	7.04e−05	89.70	14.10	202.50	2.26×	202.60	2.26×
	2000	7.68e−04	1.32e−03	95.80	19.20	217.00	2.27×	214.60	2.24×

by G-RANS, we run each Krylov solver until it attains the same *per-instance relative residual* achieved by G-RANS on that instance, i.e., we set its stopping threshold to $\text{rtol}_i = \|r_T^{\text{G-RANS}}(i)\|_2 / \|r_0(i)\|_2$.

4.2. Evaluation Protocol

Accuracy metrics. We use the standard solver metric, *relative residual*, as the primary measure ($\|r_T\|_2 / \|r_0\|_2$, where $r_k = b - Ax_k$ and $x_0 = 0$). We additionally report the absolute residual norm $\|r_T\|_2$ as a secondary diagnostic.

G-RANS inference. G-RANS is executed as a multi-step solver with early stopping. We terminate an instance once the residual falls below a prescribed tolerance or when stagnation is detected by consistently insufficient per-step residual reduction ($\leq 5\%$ in our settings); otherwise, we stop at a fixed maximum number of iterations.

Timing. All methods are benchmarked on an NVIDIA H100 GPU with synchronized timing, reporting per-instance wall-clock time in milliseconds. For G-RANS, we separately report *model time* (network forward pass) and *total time* (end-to-end solver runtime).

4.3. Experiments Results

Accuracy. We evaluate G-RANS and end-to-end neural operator baselines (GNO and FNO) that are trained under the same experimental settings. Table 3 shows that on $n \approx 1000$ systems, the operator baselines typically plateau at relative residuals around 10^{-2} and degrade to the 10^{-1} level in more challenging cases such as Helmholtz. In contrast, G-RANS consistently reduces the relative residual to the 10^{-5} regime across all PDE types.

Generalization. G-RANS remains stable under large coefficient shifts in Fig. 4 and maintains high-accuracy and similar wall-clock time even at $\pm 70\%$ variation in Table 2. It also generalizes across equation types within this second-order elliptic class, achieving high-accuracy on PDE families unseen during training (Table 4; Appendix Table 9).

Efficiency. Within the relative-residual regime reached by G-RANS, it is consistently faster than classical Krylov solvers. Across problem sizes and parameter variations, it typically achieves 2–4× wall-clock speedups over GMRES,

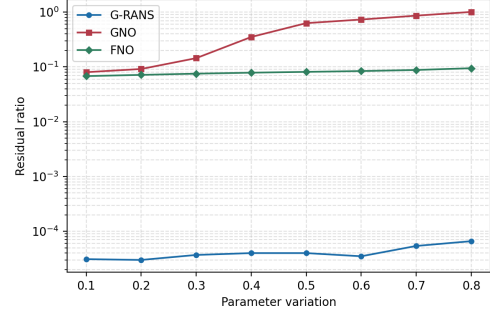


Figure 4. Parameter generalization on Poisson systems with $n \approx 1000$. The plot shows the relative residual versus coefficient variation for G-RANS and GNO/FNO.

with similar gains against FGMRES and MINRES when applicable (Table 2; Appendix Table 8). The advantage is often more pronounced at larger dimensions and stronger coefficient shifts. The reported *total* runtime includes all linear-algebraic steps (e.g., orthogonalization and projection), while the network forward pass accounts for only a fraction of the overall cost.

Compatibility. G-RANS is fully compatible with standard preconditioning. Under left Jacobi preconditioning, it still delivers 2–3× wall-clock speedups over both GMRES and FGMRES on the reaction–diffusion and Helmholtz systems (Table 5). The speedup is moderately reduced relative to the unpreconditioned setting, which may be because preconditioning lowers the cost of the early, easier residual-reduction regime, narrowing the time gap.

4.4. Ablation Study

We ablate two configurations that are central to making G-RANS *residual-aware* in an iterative solver: **multi-scale** subspace prediction in the Residual-Aware Basis Generator (RBG) and **multi-stage** progressive bootstrap training.

In ablation experiments, **–Multi-scale** replaces the multi-scale RBG with a GATv2 backbone of comparable parameter count, emitting all k basis directions only from the final layer. **–Multi-stage** trains the same architecture using only the single-stage objective (one-step residual projection), without progressive bootstrap.

We also report **sub-NSS**, which removes both mechanisms,

Table 6. G-RANS scalability and 3D extension results under $\pm 10\%$ parameter variations. We report the mean $\pm$ standard deviation for neural iterations (Iters), final relative residual (RelRes), and the wall-clock speedup rates over GPU-benchmarked classical solvers ($\text{Spd}_{\text{gmres}}$ and $\text{Spd}_{\text{fgmres}}$).

PDE	n	Iters	RelRes	$\text{Spd}_{\text{gmres}}$	$\text{Spd}_{\text{fgmres}}$
Poisson-2D	≈ 10000	19.00 ± 0.00	$(8.09 \pm 0.73)\text{e-}04$	$2.61\times$	$2.35\times$
Helmholtz-2D	≈ 10000	15.88 ± 0.93	$(9.71 \pm 2.74)\text{e-}04$	$2.79\times$	$2.62\times$
Poisson-3D	≈ 1000	21.06 ± 0.71	$(3.75 \pm 2.95)\text{e-}04$	$2.36\times$	$2.32\times$
Helmholtz-3D	≈ 1000	20.88 ± 1.20	$(6.99 \pm 4.56)\text{e-}04$	$2.43\times$	$2.40\times$
Poisson-3D	≈ 10000	7.22 ± 3.79	$(3.84 \pm 2.17)\text{e-}03$	$1.14\times$	$1.13\times$
Helmholtz-3D	≈ 10000	7.44 ± 3.11	$(5.41 \pm 3.46)\text{e-}03$	$1.17\times$	$1.16\times$

Table 7. Ablation on $n \approx 1000$ Poisson systems ($\pm 10\%$ parameter variation).

Ablation	Iters	RelRes	$\ r_T\ _2$
G-RANS	23.41	$2.80\text{e-}5$	$1.7585\text{e-}4$
–Multi-Stage	23.23	$5.22\text{e-}4$	$3.3599\text{e-}3$
–Multi-Scale	8.80	$2.47\text{e-}2$	$1.5488\text{e-}1$
sub-NSS	8.73	$3.09\text{e-}2$	$1.9127\text{e-}1$

as a lightweight baseline for neural iterative solving. As shown in Table 7, removing multi-stage training leads to noticeably weaker late-iteration progress, consistent with stagnation under residual distribution shift, whereas removing the multi-scale design substantially degrades the quality of residual-aware subspace generation, resulting in much larger final residuals.

Furthermore, we investigate the sensitivity of the solver to the maximum active subspace dimension $K_{\max}$ (see 2.4). Empirical evaluations indicate that varying $K_{\max}$ within a reasonable operational range (e.g., between 200 and 300) does not result in strong monotonic performance degradation.

4.5. Scalability and Higher-Dimensional Extensions

To evaluate the limits of our proposed framework beyond the standard small-scale 2D benchmarks, we investigate the performance of G-RANS on larger mesh resolutions ($n \approx 10^4$) and higher-dimensional physical domains (3D sparse FEM systems). For the large-scale 2D tasks, we scale up the model capacity by increasing the number of attention heads and hidden dimensions, while unrolling the progressive bootstrap schedule to 7 training stages.

As summarized in Table 6, G-RANS scales robustly to the $n \approx 10^4$ regime for both 2D Poisson and Helmholtz equations, sustaining solid relative residuals below 1.0×10^{-3} and consistently yielding $2.3\times$ to $2.8\times$ wall-clock speedups over competitive Krylov baselines.

Furthermore, we extend G-RANS to 3D elliptic systems.

At the $n \approx 10^3$ scale in 3D, the solver maintains its performance edge, achieving a relative residual of approximately 3.75×10^{-4} with over a $2.3\times$ speedup margin. When pushing to 3D systems at the $n \approx 10^4$ scale, G-RANS remains fundamentally applicable and outpaces classical iterations, though the efficiency gains compress slightly ($\approx 1.15\times$). We attribute this to sub-optimal hyperparameter configuration in this challenging regime, and we view the fine-tuning of multi-scale graph architectures for massive 3D systems as a promising avenue for follow-up work.

5. Conclusion and Outlook

We proposed G-RANS, a residual-aware neural subspace solver for large sparse linear systems arising from PDE discretization. Experiments on sparse FEM systems from representative second-order elliptic PDE families show that G-RANS reaches the 10^{-5} relative-residual regime with strong robustness to coefficient shifts and cross-equation generalization, and is faster than classical Krylov solvers within this accuracy range while remaining compatible with standard preconditioning.

Looking forward, improving the basis generator beyond the current GMRES-order instantiation may enable tighter residual regimes within the same projection-based framework. In addition, the cross-equation results in Table 4 (React.-Diff. $\rightarrow$ Poisson) suggest a data-efficient transfer phenomenon, where training on certain representative equation families can generalize so well that it outperforms training directly on the target family for some cases.

Impact Statement

We present a foundational methodological advance in AI for Science, specifically for accelerating the solution of sparse linear systems arising in PDE discretizations. By significantly reducing the computational time required for high-fidelity simulations compared to classical solvers, our approach holds the potential to accelerate discovery and design cycles in engineering and science. There are no direct negative societal impacts anticipated from this work.

References

- Brody, S., Alon, U., and Yahav, E. How attentive are graph attention networks? In *International Conference on Learning Representations*, 2022.
- Chen, J. Graph neural preconditioners for iterative solutions of sparse linear systems. In *International Conference on Learning Representations*, 2025.
- Dimola, N. and Coclite, A. Neural preconditioning via krylov subspace geometry. *Bollettino dell’Unione Matematica Italiana*, 12 2025. ISSN 2198-2759.
- Eshaghi, M. S., Anitescu, C., Valizadeh, N., Wang, Y., Zhuang, X., and Rabczuk, T. Nows: Neural operator warm starts for accelerating iterative solvers. *preprint arXiv:2511.02481*, 2025.
- Golub, G. H. and Van Loan, C. F. *Matrix Computations*. Johns Hopkins University Press, 4 edition, 2013.
- Häusner, P., Juscafresa, A. N., and Sjölund, J. Learning incomplete factorization preconditioners for GMRES. In *Northern Lights Deep Learning Conference*, 2025.
- Kaneda, A., Akar, O., Chen, J., Kala, V. A. T., Hyde, D., and Teran, J. A deep conjugate direction method for iteratively solving linear systems. In *International Conference on Machine Learning*, 2023.
- Kovachki, N., Li, Z., Liu, B., Azizzadenesheli, K., Bhattacharya, K., Stuart, A., and Anandkumar, A. Neural operator: Learning maps between function spaces with applications to pdes. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- Li, Y., Chen, P. Y., Du, T., and Matusik, W. Learning preconditioner for conjugate gradient pde solvers. *preprint arXiv:2305.16432*, 2023.
- Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. Neural operator: Graph kernel network for partial differential equations. *arXiv preprint arXiv:2003.03485*, 2020.
- Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.
- Li, Z., Xiao, D., Lai, Z., and Wang, W. Neural preconditioning operator for efficient pde solves. *preprint arXiv:2502.01337*, 2025.
- Luo, J., Wang, J., Wang, H., Dong, H., Geng, Z., Chen, H., and Kuang, Y. Neural krylov iteration for accelerating linear system solving. In *Advances in Neural Information Processing Systems*, volume 37, pp. 128636–128667, 2024.
- Manteuffel, T. A. An incomplete factorization technique for positive definite linear systems. *Mathematics of computation*, 34(150):473–497, 1980.
- Paige, C. C. and Saunders, M. A. Solution of sparse indefinite systems of linear equations. *SIAM journal on numerical analysis*, 12(4):617–629, 1975.
- Rudikov, A., Fanaskov, V., Muravleva, E., Laevsky, Y. M., and Oseledets, I. Neural operators meet conjugate gradients: The FCG-NO method for efficient PDE solving. In *International Conference on Machine Learning*, 2024.
- Ruge, J. W. and Stüben, K. Algebraic multigrid. In *Multigrid methods*, pp. 73–130. SIAM, 1987.
- Saad, Y. A flexible inner-outer preconditioned gmres algorithm. *SIAM Journal on Scientific Computing*, 14(2):461–469, 1993.
- Saad, Y. *Iterative methods for sparse linear systems*. SIAM, 2003.
- Saad, Y. and Schultz, M. H. Gmres: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on scientific and statistical computing*, 7(3):856–869, 1986.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. Graph attention networks. In *International Conference on Learning Representations*, 2018.
- Watkins, D. S. *The Matrix Eigenvalue Problem: GR and Krylov Subspace Methods*, volume 101 of *Other Titles in Applied Mathematics*. SIAM, Philadelphia, PA, 2007. ISBN 978-0-898716-41-2.

A. Related Work

Classical Iterative Solvers and Preconditioning. The standard approach for solving large-scale sparse linear systems relies on Krylov subspace methods, which iteratively construct an approximate solution within the subspace spanned by the powers of the matrix applied to the initial residual (Saad, 2003). For general non-symmetric systems, the Generalized Minimal Residual (GMRES) method (Saad & Schultz, 1986) is the canonical choice, minimizing the Euclidean norm of the residual vector over the Krylov subspace at each step. To accommodate variable preconditioners or inner-outer iteration schemes, Flexible GMRES (FGMRES) (Saad, 1993) relaxes the requirement of a constant preconditioner. In the case of symmetric indefinite systems, the Minimum Residual (MINRES) method (Paige & Saunders, 1975) is often preferred, as it exploits symmetry to minimize the residual using short recurrences, significantly reducing memory overhead. To improve the spectral properties of the linear operator and accelerate convergence, these solvers are typically coupled with preconditioners. Common techniques range from simple stationary methods like the Jacobi preconditioner (diagonal scaling) (Watkins, 2007) to more complex algebraic multigrid (Ruge & Stüben, 1987) or incomplete factorization schemes (Manteuffel, 1980).

Fixed Subspace Initialization and Neural Warm Starts. Recent works integrate networks with Krylov solvers by improving their initialization rather than modifying the iterative procedure itself. Existing methods, e.g., NeurKItt (Luo et al., 2024), predict approximate invariant or low-frequency subspaces using neural networks and use them to initialize Krylov iterations. Once constructed, these subspaces remain fixed throughout solving. Similarly, neural operator-based warm-start approaches like Nows (Eshaghi et al., 2025) aim to generate a coarse initial solution that is subsequently refined by classical iterative solvers. These methods decouple neural inference from the iterative loop and primarily target faster early convergence.

Learned Preconditioning for Krylov Methods. Many works accelerate Krylov methods by learning preconditioning operators. Neural networks are used to construct operators that act on residuals or search directions over iteration, implicitly improving the spectral properties of the effective system while preserving standard Krylov subspace construction. Representative examples draw inspiration from classical preconditioning techniques such as algebraic multigrid (GNP(Chen, 2025) and NPO(Li et al., 2025)), incomplete LU factorizations (Häusner et al., 2025), and conjugate gradients (FCG-NO(Rudikov et al., 2024), DCDM(Kaneda et al., 2023) and NeuralPCG(Li et al., 2023)), as well as approaches that incorporate Krylov subspace geometry into the learning process (NP-KSG) (Dimola & Coclite, 2025).

B. Extended Background on GMRES

In Section 3, we established that G-RANS generalizes the GMRES algorithm under the Linear Identity Configuration. To elucidate this connection, we provide a detailed overview of the Generalized Minimal Residual (GMRES) method (Saad & Schultz, 1986) and present its algorithmic structure in a format comparable to G-RANS.

GMRES belongs to the family of Krylov subspace methods. Given a linear system $\mathbf{Ax} = \mathbf{b}$ and an initial guess $\mathbf{x}_0$, it seeks an approximate solution $\mathbf{x}_m$ from the affine subspace $\mathbf{x}_0 + \mathcal{K}_m$, where $\mathcal{K}_m = \text{span}\{\mathbf{r}_0, \mathbf{Ar}_0, \dots, \mathbf{A}^{m-1}\mathbf{r}_0\}$ is the Krylov subspace of order m .

The algorithm consists of two primary components that map directly to the G-RANS framework:

1. **Basis Generation (Arnoldi Process):** This constructs an orthonormal basis $\mathbf{V}_m$ for $\mathcal{K}_m$. In the context of G-RANS, this corresponds to the *Residual-Aware Basis Generator (RBG)* combined with the QR factorization. While RBG uses a learned non-linear graph neural network to predict directions, GMRES uses a deterministic matrix-vector product $\mathbf{Av}_j$ followed by Gram-Schmidt orthogonalization.
2. **Subspace Projection:** GMRES computes the minimizer of the residual norm $\|\mathbf{b} - \mathbf{Ax}\|_2$ within the constructed subspace. This is mathematically equivalent to the *Differentiable Numerical Subspace Projection* (Eq. 11) in G-RANS, solving a least-squares problem involving the matrix generated by the Arnoldi process.

Algorithm 2 outlines the standard GMRES procedure. Note the structural alignment with G-RANS (Algorithm 1).

C. Data Generation Details

Data generation. We generate sparse linear systems $\mathbf{Ax} = \mathbf{b}$ by discretizing PDEs with a finite-element method (FEM) on triangular meshes. The computational domain is a quadrilateral with vertices $(-1, -1)$, $(1, -1)$, $(1, 1)$, and $(-1, 1)$; to

Algorithm 2 Restarted GMRES (GMRES(m))

```

1: Input:  $\mathbf{A}$ ,  $\mathbf{b}$ ,  $\mathbf{x}_0$ , restart  $m$ , tol  $\epsilon$ .
2:  $\mathbf{x} \leftarrow \mathbf{x}_0$ .
3: while  $\|\mathbf{b} - \mathbf{Ax}\|_2 \geq \epsilon$  do
4:    $\mathbf{r}_0 \leftarrow \mathbf{b} - \mathbf{Ax}$ ,  $\beta \leftarrow \|\mathbf{r}_0\|_2$ ; if  $\beta = 0$  break.
5:    $\mathbf{v}_1 \leftarrow \mathbf{r}_0/\beta$ , set  $\mathbf{V}_1 = [\mathbf{v}_1]$ .
6:   for  $j = 1$  to  $m$  do
7:      $\mathbf{w} \leftarrow \mathbf{Av}_j$  (SpMV; new Krylov direction)
8:     for  $i = 1$  to  $j$  do
9:        $h_{ij} \leftarrow \mathbf{v}_i^T \mathbf{w}$ ;  $\mathbf{w} \leftarrow \mathbf{w} - h_{ij} \mathbf{v}_i$  (MGS orthogonalization; cf. block re-ortho)
10:    end for
11:     $h_{j+1,j} \leftarrow \|\mathbf{w}\|_2$ ; if  $h_{j+1,j} = 0$  break (breakdown)
12:     $\mathbf{v}_{j+1} \leftarrow \mathbf{w}/h_{j+1,j}$ ;  $\mathbf{V}_{j+1} \leftarrow [\mathbf{V}_j, \mathbf{v}_{j+1}]$ .
13:    Build  $\bar{\mathbf{H}}_j \in \mathbb{R}^{(j+1) \times j}$  from  $\{h_{is}\}$ .
14:     $\mathbf{y}_j \leftarrow \arg \min_{\mathbf{y}} \|\beta \mathbf{e}_1 - \bar{\mathbf{H}}_j \mathbf{y}\|_2$  (least-squares projection)
15:     $\mathbf{x} \leftarrow \mathbf{x} + \mathbf{V}_j \mathbf{y}_j$  (update)
16:    if  $\|\mathbf{b} - \mathbf{Ax}\|_2 < \epsilon$  then
17:      break
18:    end if (stop)
19:  end for
20: end while
21: return  $\mathbf{x}$ .

```

introduce mild geometric variability, the two top corners may be perturbed. We impose a Dirichlet boundary condition on the bottom edge ($y = -1$) by default across all benchmark experiments to maintain consistency. However, because G-RANS operates directly on the resulting algebraic sparse system $\mathbf{Ax} = \mathbf{b}$, other boundary conditions—such as Neumann or mixed boundary conditions—are not fundamentally restricted and can be applied as needed, as they are naturally absorbed into the sparse matrix structure $\mathbf{A}$ and the right-hand side vector $\mathbf{b}$.

Meshes are generated with a target node count n (the number of unknowns after discretization), with slight deviations due to meshing. All PDE families share the same meshing pipeline, and we consider two problem sizes with $n \in \{1000, 2000\}$. For each instance, the FEM solver outputs the solution vector $\mathbf{x}$, the sparse stiffness matrix $\mathbf{A}$ (stored in COO format), and the right-hand side $\mathbf{b}$; node coordinates are retained as graph features. When preconditioning is enabled (only in the compatibility study), we apply left Jacobi preconditioning, i.e., $(D^{-1}\mathbf{A})\mathbf{x} = D^{-1}\mathbf{b}$, where $D = \text{diag}(\mathbf{A})$.

Coefficient randomization. For each instance, PDE coefficients are randomized independently via multiplicative perturbations. For any scalar coefficient p , we sample

$$p = p_0(1 + u), \quad u \sim \mathcal{U}[-\nu, \nu],$$

where $\nu \in \{0.1, 0.3, 0.7\}$ is the variation ratio. Spatially varying coefficients are constructed as piecewise-constant fields defined by a circular inclusion embedded in a homogeneous background. The inclusion radius and center are mildly randomized as

$$r = r_0(1 + u_r), \quad u_r \sim \mathcal{U}[-0.5\nu, 0.5\nu], \quad \mathbf{c} = \mathbf{c}_0 + \delta\mathbf{c}, \quad \delta\mathbf{c} \sim \mathcal{U}[-0.2\nu, 0.2\nu] \text{ (per axis)}.$$

This yields coefficients of the form

$$\kappa(\mathbf{x}) = \begin{cases} \kappa_{\text{in}}, & \|\mathbf{x} - \mathbf{c}\| \leq r, \\ \kappa_{\text{out}}, & \text{otherwise,} \end{cases} \quad \alpha(\mathbf{x}) = \begin{cases} \alpha_{\text{in}}, & \|\mathbf{x} - \mathbf{c}\| \leq r, \\ \alpha_{\text{out}}, & \text{otherwise.} \end{cases}$$

The source magnitude and boundary scaling (when applicable) are randomized using the same multiplicative scheme. When geometry randomization is enabled, we additionally perturb the two top corners by i.i.d. offsets from $\mathcal{U}[-0.3, 0.3]$, inducing mild domain-shape variation across samples.

Specific PDE Formulations. Unless stated otherwise, the default data distribution used for training sets is $\nu = 0.1$ ($\pm 10\%$ parameter variation). Below we summarize the specific governing equations and nominal parameters for each PDE family. All systems are subject to the boundary scaling $u|_{\partial\Omega} = g(\mathbf{x})$.

2D Poisson Equation. The steady-state diffusion process is defined as:

$$-\nabla \cdot (\kappa(\mathbf{x}) \nabla u(\mathbf{x})) = f(\mathbf{x}), \quad \mathbf{x} \in \Omega$$

- **Diffusion coefficient:** Nominal values are $\kappa_{\text{in}} = 1.1$ and $\kappa_{\text{out}} = 1.0$, yielding randomized ranges of $[0.99, 1.21]$ and $[0.9, 1.1]$.
- **Source term:** Constant source field $f(\mathbf{x})$ with a nominal value of 0.1, varying within $[0.09, 0.11]$.
- **Boundary scale:** The Dirichlet boundary function $g(\mathbf{x})$ is scaled by a multiplier randomized within $[0.9, 1.1]$.
- **Inclusion Geometry:** The perturbed radius r ranges within $[0.2375, 0.2625]$, and center offsets $\delta\mathbf{c}$ fluctuate within ± 0.01 per axis.

2D Advection-Diffusion Equation. To evaluate non-symmetric system matrices, we incorporate a linear velocity field:

$$-\nabla \cdot (\kappa(\mathbf{x}) \nabla u(\mathbf{x})) + \mathbf{v}(\mathbf{x}) \cdot \nabla u(\mathbf{x}) = f(\mathbf{x}), \quad \mathbf{x} \in \Omega$$

where $\kappa(\mathbf{x})$, $f(\mathbf{x})$, and the boundary constraints inherit the exact nominal setups and variation intervals from the Poisson suite. The velocity field models a steady vortex flow:

$$\mathbf{v}(\mathbf{x}) = s \cdot (-y, x)$$

Under the default $\pm 10\%$ perturbation, the nominal scaling factor $s_0 = 0.5$ is randomized within $[0.45, 0.55]$.

2D Helmholtz Equation. For challenging indefinite sparse systems, we evaluate the Helmholtz equation:

$$-\nabla \cdot (\kappa(\mathbf{x}) \nabla u(\mathbf{x})) + \alpha(\mathbf{x}) u(\mathbf{x}) = f(\mathbf{x}), \quad \mathbf{x} \in \Omega$$

where $\kappa(\mathbf{x})$ and $f(\mathbf{x})$ follow the Poisson configuration. The reaction/wave-number coefficient $\alpha(\mathbf{x})$ uses the circular inclusion parameterization:

- **Reaction coefficient:** Nominal values are $\alpha_{\text{in}} = 2.0$ and $\alpha_{\text{out}} = 1.0$, giving randomized intervals of $[1.8, 2.2]$ and $[0.9, 1.1]$.
- **Inclusion Geometry:** The center shift $\delta\mathbf{c}$ is allowed broader perturbation, fluctuating within ± 0.02 per axis.

2D Reaction-Diffusion Equation. We benchmark steady-state reaction-diffusion mechanics involving highly damped zero-order interactions:

$$-\nabla \cdot (\kappa(\mathbf{x}) \nabla u(\mathbf{x})) + \sigma(\mathbf{x}) u(\mathbf{x}) = f(\mathbf{x}), \quad \mathbf{x} \in \Omega$$

The spatial variations for the absorption parameter $\sigma(\mathbf{x})$ (substituting $\alpha(\mathbf{x})$ in the piecewise definition) are:

- **Absorption coefficient:** Nominal values are $\sigma_{\text{in}} = 3.0$ and $\sigma_{\text{out}} = 1.5$, yielding randomized ranges of $[2.7, 3.3]$ and $[1.35, 1.65]$.

D. Additional Experiments

The additional experiment results are displayed in Tab. 8 and 9.

Table 8. G-RANS results on $n \approx 2000$ sparse FEM systems under parameter variations. We report mean $\pm$ std neural iterations, final relative residual ratio $\|r_T\|_2/\|r_0\|_2$ (with $r_0 = b$), absolute final residual norm $\|r_T\|_2$, per-instance wall-clock time (total and model-only), and speedup of classical solvers measured in the accuracy reached by G-RANS (baseline time divided by G-RANS total time). MINRES is only applicable to symmetric systems and thus is not reported for non-symmetric cases.

PDE	Var.	Iters	RelRes	$\ r_T\ _2$	t_{tot}	t_{model}	Spd _{gmres}	Spd _{fgmres}	Spd _{minres}
Poisson	$\pm 10\%$	23.44	3.42e-04	2.1935e-03	105.00	21.80	4.48×	4.06×	3.12×
	$\pm 30\%$	22.92	5.19e-04	3.3377e-03	103.60	22.00	4.21×	3.81×	3.23×
	$\pm 70\%$	22.86	5.35e-04	3.2548e-03	102.80	21.90	4.98×	3.71×	4.06×
Adv.-Diff.	$\pm 10\%$	22.17	7.35e-04	4.6385e-03	100.60	23.40	3.65×	3.38×	—
	$\pm 30\%$	22.08	7.40e-04	4.3063e-03	98.70	23.20	3.86×	3.45×	—
	$\pm 70\%$	21.45	8.85e-04	5.5851e-03	100.30	24.10	4.34×	3.44×	—
Helmholtz	$\pm 10\%$	23.33	4.79e-04	4.3062e-03	106.80	24.70	3.35×	2.84×	—
	$\pm 30\%$	23.34	5.55e-04	5.3717e-03	109.70	25.70	3.03×	2.58×	—
	$\pm 70\%$	23.14	9.14e-04	6.9664e-03	106.00	24.40	3.03×	2.37×	—
React.-Diff.	$\pm 10\%$	23.62	3.56e-04	3.1850e-03	110.50	25.30	3.08×	2.60×	3.33×
	$\pm 30\%$	23.66	4.01e-04	3.3037e-03	110.60	25.20	2.94×	2.49×	3.18×
	$\pm 70\%$	23.50	6.05e-04	4.8026e-03	110.40	25.60	3.03×	2.33×	3.52×

Table 9. Cross-equation generalization of G-RANS on $n \approx 2000$ systems under $\pm 10\%$ parameter variation. Rows indicate the training PDE and columns indicate the test PDE. Values are the final relative residual $\|r_T\|_2/\|r_0\|_2$.

Train \ Test	Poisson	Adv.-Diff.	Helmholtz	React.-Diff.
Poisson	3.4×10^{-4}	4.2×10^{-4}	6.1×10^{-4}	4.0×10^{-4}
Adv.-Diff.	6.7×10^{-4}	7.4×10^{-4}	1.0×10^{-3}	7.0×10^{-4}
Helmholtz	7.8×10^{-4}	6.8×10^{-4}	4.8×10^{-4}	3.2×10^{-4}
React.-Diff.	8.1×10^{-4}	7.9×10^{-4}	5.7×10^{-4}	3.6×10^{-4}